

Documentación de Arquitectura — App IP ASEA (Verde Raíz)

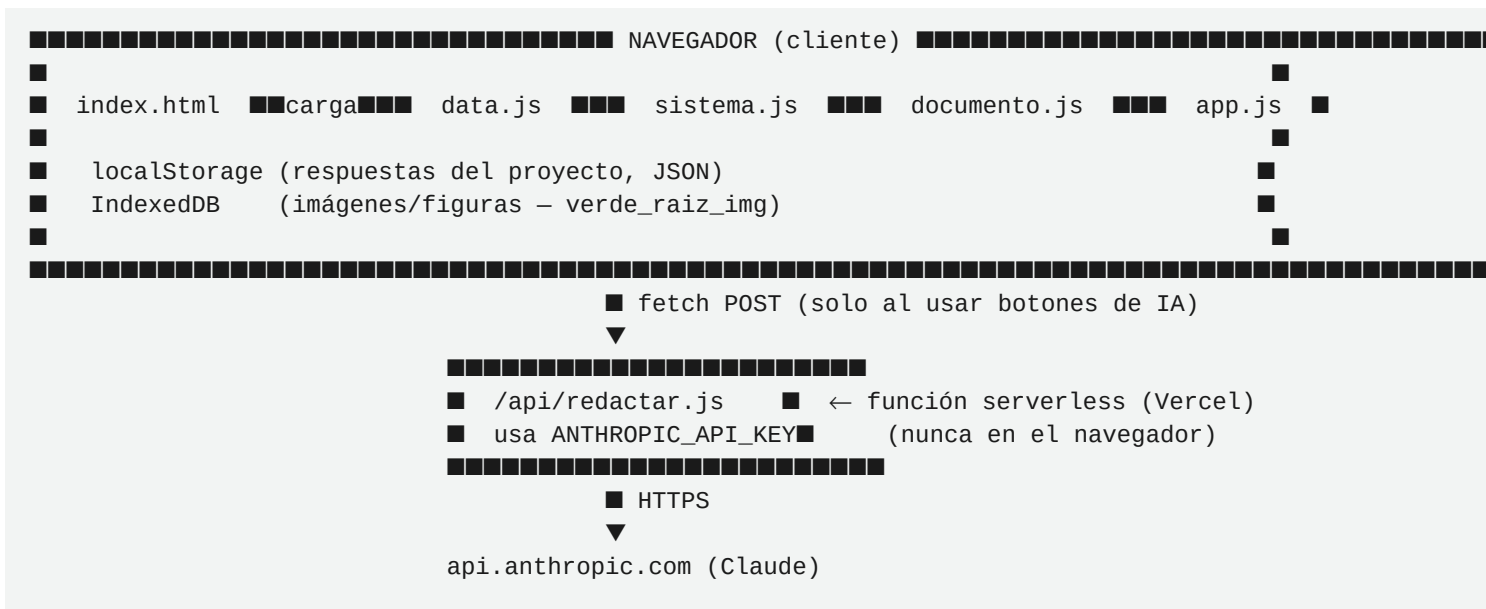
Verde Raíz
Julio 2026

1. Propósito de este documento

Describe qué hace cada módulo del sistema, cómo se comunican entre sí y cómo desplegarlo/mantenerlo, para que un desarrollador externo (soporte técnico, freelancer, o el propio equipo en el futuro) pueda dar mantenimiento sin depender del autor original.

2. Visión general

Es una aplicación **web estática de un solo archivo HTML** (sin framework, sin build, sin `package.json`) más **una función serverless** que hace de puente seguro hacia el proveedor de IA. No hay base de datos ni backend con estado: todo el estado del proyecto vive en el navegador del usuario.



No hay llamadas a ningún otro servicio propio: la única dependencia externa en tiempo de ejecución es la API de Anthropic (y, opcionalmente, la exportación pública de un Google Sheet cuando el usuario pega un link — ver Documento 3).

3. Módulos del cliente (carpeta raíz)

3.1 `index.html`

El *shell* de la aplicación: layout (header, barra lateral con la tabla de contenido, panel del formulario, panel de vista previa del documento), estilos (`<style>` embebido, sin CSS externo) y las etiquetas `<script>` que cargan, en orden, `data.js` → `sistema.js` → `documento.js` → `app.js`. No tiene lógica propia más allá de la estructura del DOM sobre la que `app.js` escribe.

3.2 `data.js` — catálogos y boilerplate

Expone un único objeto global `window.IPDATA` con ~35 catálogos y bloques de texto fijo. Es la fuente de verdad de "todo lo que no cambia entre proyectos, o cambia dentro de un conjunto cerrado de opciones":

Categoría	Ejemplos de claves
Datos fijos de la consultora	<code>CONSULTORA</code>
Catálogo de sustancias (CRETIB/CAS)	<code>SUSTANCIAS</code> , <code>SUS_CONSTRUCCION_DEFAULT</code> , <code>SUS_OPERACION_DEFAULT</code>
Valores por defecto de ~25 tablas técnicas	<code>RESIDUOS_DEFAULT</code> , <code>CLIMA_DEFAULT</code> , <code>RECEPTORES_DEFAULT</code> , <code>ACCIONES_PROYECTO_DEFAULT</code> , <code>GATILLOS_DEFAULT</code> , etc.
Texto boilerplate nacional	<code>BOILER</code> (introducción, POEGT, PND, MEC, metodología Leopold/Gómez-Orea, parque industrial)
44 estrategias POEGT con respuesta estándar	<code>ESTRATEGIAS_POEGT</code> , <code>DISPOSICIONES</code>
Catálogos estatales	<code>ESTADOS</code> , <code>LISTA_ESTADOS</code> , <code>CRITERIOS_UGA_JAL</code> , <code>CRITERIOS_UGA_HID</code>
Instrumentos jurídicos	<code>INSTRUMENTOS</code>
Catálogo de figuras/mapas esperados por sección	<code>AREAS</code> (33 huecos de imagen, ver Documento 3 §4)
Referencias bibliográficas fijas	<code>REFERENCIAS</code>

Para agregar/editar un catálogo (p. ej. un nuevo estado con su Plan Estatal y criterios UGA): se edita directamente este archivo — no requiere tocar `app.js` salvo que el nuevo catálogo necesite un tipo de campo nuevo (ver §3.4).

3.3 `sistema.js` — contenido embebido de guía y checklist

Carga después de `data.js` y **agrega** más claves al mismo `window.IPDATA`:

- `GUIA_HTML` — el texto de la "■ Guía de uso" que se muestra dentro de la propia app.
- `CHECKLIST_ASEA` — los ítems del checklist ASEA-00-041 que el usuario marca antes de dar por cerrada una sección.
- `ANEXOS_DOCS` — el catálogo de documentos requeridos en el capítulo VI (acta constitutiva, poder notarial, planos, dictámenes...) con su link editable.

- `JURIDICO` — contenido de referencia legal usado en II.1.

Existe como archivo separado por convención editorial (para no mezclar "catálogos técnicos" con "contenido de ayuda/checklist"), pero técnicamente es una extensión de `data.js`.

3.4 `app.js` — cuestionario, estado y motor de ensamblado del lado cliente

El archivo más grande y el corazón de la interacción. Responsabilidades:

- **SECTIONS** — arreglo que define las 17 secciones del cuestionario (grupo, título, descripción, lista de campos). Cada campo tiene `id`, etiqueta (`l`), tipo y una insignia (`b`: cerrada / boiler / abierta). Es la definición declarativa de todo el formulario.
- **state** — objeto único con todas las respuestas del proyecto. Se persiste en `localStorage` (clave `verde_raiz_ip_v1`) en cada cambio (`save()`), y se normaliza al cargar (`normalize()`) para garantizar que arrays/objetos esperados siempre existan aunque el JSON cargado sea antiguo o esté incompleto.
- **renderForm() / renderField()** — motor de renderizado: recorre `SECTIONS` y, según el tipo de cada campo, genera el HTML correspondiente (`text`, `select`, `check`, `puntos`, `sustancias`, `susDinamica`, `estrategias`, `instrumentos`, `open`, `ia`, `tablaIA`, `importar`, `programas`, `checklist`, `anexos`, `guia...`). Cada tipo tiene su propia función `render*()` y, si necesita interacción, una función `bind*()` que conecta los elementos del DOM con `state`.
- **Bucket de imágenes (IndexedDB)** — `idbOpen/idbPut/idbDel/idbAll`, más `fileToDataURL()` que redimensiona la imagen (máx. 1600 px) antes de guardarla. Permite cientos de MB de fotos/planos sin saturar `localStorage` (que tiene límite de ~5–10 MB).
- **Multi-proyecto** — `newProject/openProject/deleteProject/saveCurrentProject/loadIdx`: la app mantiene un índice de proyectos y permite tener varios en paralelo, cada uno con su propio `state` e imágenes.
- **Integraciones de IA (clientes del backend)** — tres flujos que llaman a `/api/redactar` (ver §4):
 - `bindImportar()` → `accion:"perfil"` (autollenado de la recopilación).
 - `bindTablaIA()` → `accion:"tabla"` (estructurar tablas técnicas).
 - `bindProgramas()` → `accion:"programa"` (N programas de ordenamiento, §3.4 de este documento aplica también aquí).
 - `bindIA()` → sin `accion` explícita, usa el `seccion` correspondiente (ver Documento 4) para redactar prosa técnica en las secciones abiertas.
- **Motor de ensamblado del documento** — `buildBlocks()` llama a `documento.js` (§3.5) con el contexto actual (`state`, catálogos, helpers), y `renderBlocks()/renderLiveDoc()` pintan el resultado en el panel de vista previa (modo Borrador/Limpio).
- **Exportación** — `exportWord(modos)` arma un documento **HTML con estilos de impresión** (`wordStyles()`) y lo descarga con `Content-Type: application/msword`; Word lo abre nativamente (no es un `.docx/OOXML` real, es HTML que Word interpreta). La exportación a PDF usa el diálogo de impresión nativo del navegador sobre esa misma vista.

3.5 documento.js — machote del IP (motor de ensamblado, puro)

Expone `window.IPDOC.build(ctx)`, una función pura que recibe el contexto (`state`, catálogos, helpers de formato) y devuelve un arreglo de **bloques** (`{t:"h"|"p"|"table"|"fig"|"cover"|"toc", k:"static"|"auto"|"instr", ...}`). No toca el DOM directamente — separa "qué dice el documento" (`documento.js`) de "cómo se pinta en pantalla o se exporta" (`app.js`).

Helpers internos relevantes para quien vaya a extender una sección:

Helper	Uso
<code>H(nivel, texto)</code>	Encabezado (nivel 1 = "I.", nivel 3 = "I.1", etc.)
<code>P(texto)</code>	Párrafo de contenido nuevo (se resalta en modo Borrador)
<code>I(texto)</code>	Instrucción/nota pendiente (se resalta siempre, en ámbar)
<code>TBL({title, head, rows, k})</code>	Tabla; <code>k</code> determina si se pinta como "auto" (llena) o "scaffold" (vacía, por llenar)
<code>FIGAREA(id)</code>	Inserta el(los) hueco(s) de imagen registrados para esa área en <code>data.js AREAS</code>
<code>tFilled/tRows/tK</code>	Helpers para las tablas alimentadas por <code>state.tablas[key]</code> (las "tablaA")
<code>IAP(key)</code>	Inserta como párrafos el texto generado por IA guardado en <code>state[key]</code>

El documento sigue el orden exacto: Portada → Índice → Glosario → Introducción → I → II (incluye el bucle de **N programas de ordenamiento**, uno por cada entrada en `state.programas`) → III → IV (Abandono) → V (Conclusión) → VI (Anexos) → Referencias.

Para agregar una subsección nueva: se agrega la llamada `H()/P()/TBL()` correspondiente en el punto exacto de `build()` donde debe aparecer; si necesita datos nuevos del usuario, esos campos se agregan primero en `SECTIONS` (`app.js`) y, si tienen que renderizarse en el Word/PDF, se leen aquí vía `g("id_del_campo")`.

4. Backend: `api/redactar.js` (función serverless)

Única pieza de backend del sistema. Se despliega como función serverless de Vercel (`module.exports = async (req, res) => {...}`), invocada por el cliente vía `POST /api/redactar`. Su razón de ser es que la `ANTHROPIC_API_KEY` **nunca puede vivir en el navegador** (se filtraría a cualquier visitante); esta función la lee

de una variable de entorno del servidor y hace la llamada a Claude en nombre del cliente.

Cuatro modos, seleccionados por el campo accion del body (o su ausencia):

acción	Función interna	Qué hace
(vacío) — trae seccion	redactarSeccion()	Redacta la prosa técnica de una de las 10 secciones abiertas del IP (ver Documento 4)
"perfil"	extraerPerfil()	Lee la recopilación (texto/Sheet/imagen) y devuelve { campos: { ... } } para autollenar el cuestionario
"tabla"	estructurarTabla()	Convierte datos pegados/imagen en filas para una o más tablaIA
"programa"	extraerPrograma()	Lee el Sheet de UN programa de ordenamiento y devuelve { campos, incisos }

Detalles de gobierno del contenido generado (reglas del SYSTEM prompt, límites de tokens, limpieza defensiva de markdown) están documentados en el **Documento 4 — Registro de Prompts**, ya que son la pieza más sensible a auditoría de calidad.

fetchSheetCSV(url) es el único punto donde el sistema sale a buscar datos de terceros: toma un link de Google Sheets, extrae el ID con una expresión regular y descarga .../export?format=csv — funciona porque el Sheet debe ser público o compartido con "cualquiera con el link"; no hay autenticación OAuth ni cuenta de servicio.

5. Despliegue

- **Plataforma:** Vercel. vercel.json declara un sitio 100% estático ("framework": null, sin buildCommand real) más la función api/redactar.js que Vercel detecta automáticamente por convención de carpeta (api/).
- **Variable de entorno requerida en producción:** ANTHROPIC_API_KEY (Vercel → Settings → Environment Variables). Sin ella, todo el cuestionario y la exportación Word/PDF funcionan igual; solo fallan los botones de IA.
- **Sin base de datos, sin autenticación de usuarios.** El "proyecto" vive enteramente en el navegador de quien lo llena; para compartir un proyecto entre dos personas se usa **Guardar JSON** → enviar el archivo → **Cargar JSON** en el otro equipo.
- **Sin proceso de build.** Cualquier cambio a app.js/data.js/documento.js/index.html se refleja con solo recargar la página; no hay paso de compilación ni dependencias de Node que instalar para desarrollar.

6. Cómo diagnosticar problemas comunes

Síntoma	Causa probable	Dónde mirar
Los botones de IA no responden / error 500	ANTHROPIC_API_KEY no configurada o inválida en Vercel	Logs de la función en el dashboard de Vercel; <code>api/redactar.js</code> línea <code>apiKey = process.env.ANTHROPIC_API_KEY</code>
"Autollenar" no encuentra nada de un Sheet	El Sheet no es público, o el link no contiene un ID de Sheets reconocible	<code>fetchSheetCSV()</code> en <code>api/redactar.js</code> (regex de extracción de ID)
Un campo nuevo no aparece en el Word exportado	Falta agregar el <code>H()/P()/TBL()</code> correspondiente en <code>documento.js</code> — el campo puede existir en <code>state</code> pero el machote no lo está leyendo	<code>documento.js</code> , buscar la sección por su número (p. ej. "I.1.4")
Las imágenes desaparecen al cambiar de equipo	Las imágenes viven en IndexedDB, que es local al navegador/equipo — no viajan solas con el JSON exportado salvo que el JSON incluya el volcado de imágenes	<code>idbAll()/Guardar JSON</code> en <code>app.js</code>
El progreso (punto de color) no refleja lo capturado	El tipo de campo nuevo no está contemplado en <code>seccionStatus()</code>	<code>app.js</code> , función <code>seccionStatus()</code>

7. Herramienta de soporte: mapa de conocimiento del código

Este repositorio tiene instalado **graphify**, una herramienta que construye un grafo navegable de todas las funciones, módulos y sus relaciones (`graphify-out/graph.json` y `graphify-out/graph.html`, ábrible en cualquier navegador). Un desarrollador externo puede usarlo para orientarse rápido sin depender de que alguien del equipo le explique la arquitectura de viva voz:

```
graphify query "cómo se genera el Word"
graphify explain "documento.js"
graphify path "app.js" "api/redactar.js"
```

`graphify-out/GRAPH_REPORT.md` trae además la lista de "nodos más conectados" (las funciones de las que depende todo lo demás) — un buen punto de partida para quien llegue nuevo al proyecto.